

AI & LLM Systems Playbook

RAG, agents, semantic caching and the backend stack behind production AI.

7 chapters · compiled from the Evolving Engineer — pradyumnachippigiri.substack.com

By Pradyumna Chippigiri

Compiled 4 October 2026

WHAT'S INSIDE

- Zero to One : Learning Agents and Agentic Patterns
- Everything you need to know about production ready RAG systems : Part 1
- Everything you need to know about Production ready RAG systems: Part 2
- Everything you need to know about production ready RAG systems : Part 3
- Semantic Caching
- A Supa Explanation of Supabase: The BaaS Everyone Is Talking About (And Why)
- Shell Scripting for Devops Beginners: Part 2

Everything you need to know about production ready RAG systems : Part 1	5
<i>From foundational building blocks to multi-agent systems: A deep dive into agentic patterns and memory in agents</i>	
Zero to One : Learning Agents and Agentic Patterns	3

CONTENTS

Zero to One : Learning Agents and Agentic Patterns

2026-01-21 · From foundational building blocks to multi-agent systems: A deep dive into agentic patterns and memory in agents

Source: <https://pradyumnachippigiri.substack.com/p/zero-to-one-learning-agents-and-agentic>

I have been working on AI Agents for some time now, and upskilling myself in this space.

As I learn I have been documenting everything from various references, and sources.

This turned into a pretty large knowledge dump covering concepts, architectures, code, and diagrams, so it didn't fit nicely into a Substack post. That's why I'm linking out for this article. Access the link below. n

LINK ? : <https://pradyumnachippigiri.dev/blogs/understanding-ai-agents>

Below is the table of contents so you can quickly see what's covered.

TABLE OF CONTENTS
Overview
What are AI Agents?
Agents vs Workflows
Agent Usecases
Building Blocks of Agents
LLMs
Tools
Memory in Agents
Short-term Memory (STM)
Long-term Memory (LTM)
Memory Management
ReACT Pattern
Agentic Frameworks
Workflow Patterns
Prompt Chaining
Routing
Parallelization
Orchestrator-Workers
Agentic Patterns
ReACT Pattern
Reflection Pattern
Tool Use Pattern
Multi Agent pattern
Multi Agent Patterns
Subagents Pattern
Skills Pattern
Handoffs Pattern
Router Pattern
More of my writing in AI
Next Steps
References

Here's a list of references and sources I have learned from:

References

- [Google Cloud Tech Youtube Videos](#)
- [Anthropic's Guide to building effective Agents](#)
- [Open AI's Practical guide to building Agents](#)
- [Microsoft: AI Agent for beginners](#)
- [IBM: Guide to Ai Agents](#)
- [Vizuara AI Labs and Dr. Raj Dandekar](#)
- [Andrew Ng: Agentic AI Course](#)
- [Phil Schmid: Rise of Subagents](#)
- [Aman: Primer on AI Agents](#)
- [Leonie Monigatti: Making sense of memory in Agents](#)
- [Cognition: Dont build multi-agents](#)
- [Simon Willison: AI Agents Definitions](#)
- [Agentlearn.dev](#)
- [Phil Schmid: Memory in AI Agents](#)
- [Weaviate: What are Agentic Workflows?](#)
- [Anthropic: Multi Agent Research System](#)
- [Langchain: Choosing the right multi-agent architectures](#)
- [DailyDoseOfDS, Avi Chawla : Ai Agents guidebook](#)
- [Lance Martin: Context Engineering for Agents](#)
- [Drew Breunig: Why Context Engineering Matters](#)
- [Tina Huang : AI Agents video](#)
- [Letta Blogs : Agent Memory, How to build Ai Agents that learn and remember](#)
- [LangChain + LangGraph Docs: Memory](#)
- [Aishwarya Naresh Reganti: LevelUpLabs free AI Agent course on Github](#)

Here's a short preview video below so you can quickly see how the article is structured.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Thanks for reading Evolving Engineer! If you know someone learning agents, feel free to share it. If this post helped, you can support my work here - Buy Me a Coffee ?

See you in the next one ?.

Everything you need to know about production ready RAG systems : Part 1

2025-11-18 · Deep-diving into document ingestion, chunking strategies, and the architecture of real-world retrieval systems

Source: <https://pradyumnachippigiri.substack.com/p/everything-you-need-to-know-about>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/rag-systems-part1>

What is RAG?

RAG stands for Retrieval Augmented Generation. It was introduced in the paper Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

Each step can be roughly broken down to:

Retrieval – Seeking relevant information from a source given a query. For example, getting relevant passages of Wikipedia text from a database given a question.

Augmented – Using the relevant retrieved information to modify an input to a generative model (e.g., an LLM).

Generation – Generating an output given an input. For example, in the case of an LLM, generating a passage of text given an input prompt.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

You can think of RAG like an open-book exam:

You or your mind is the LLM.

The textbook is your external knowledge base (PDFs, docs, webpages).

When a question is asked, you don't memorize the entire book, instead, you first flip through the index/table of contents to find the most relevant pages -> This "flipping and finding the right pages" is Retrieval.

Once you have the right pages open, you read them and augment your understanding with that information -> This is Augmentation.

Finally, based on the question + the retrieved pages, you think, synthesize, and produce a well-structured answer -> This is Generation.

So the pipeline is:

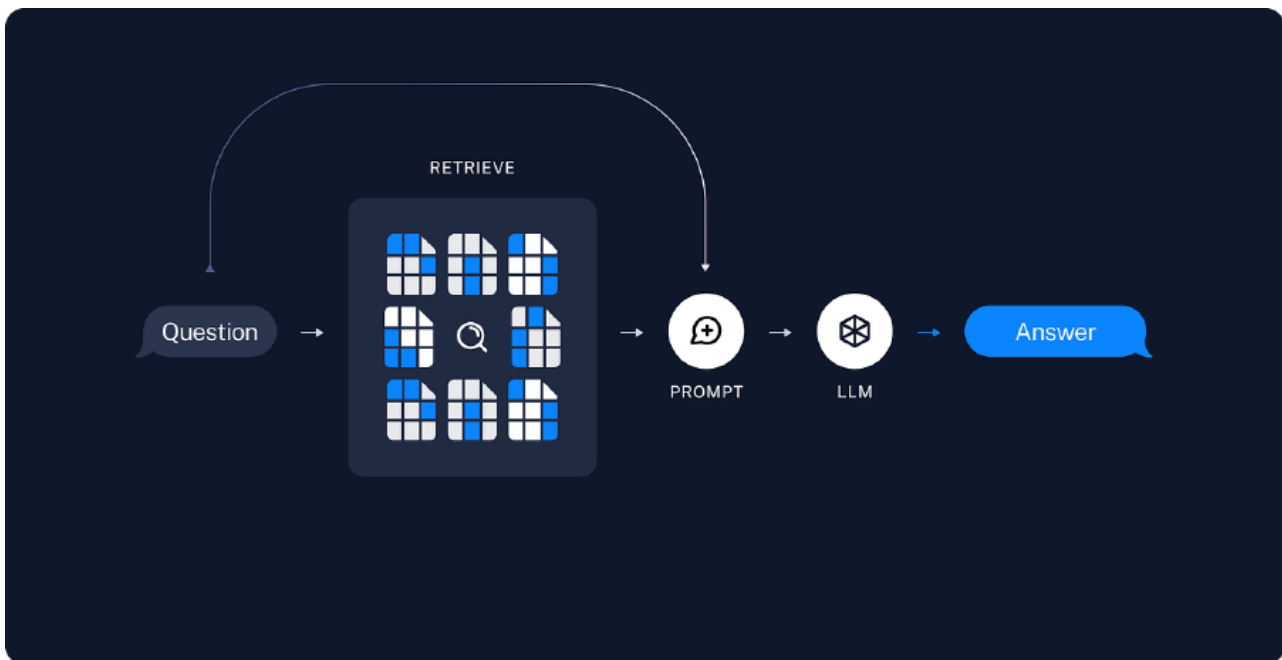
Index search -> retrieve relevant pages -> read them -> think -> answer.

Exactly how RAG works:

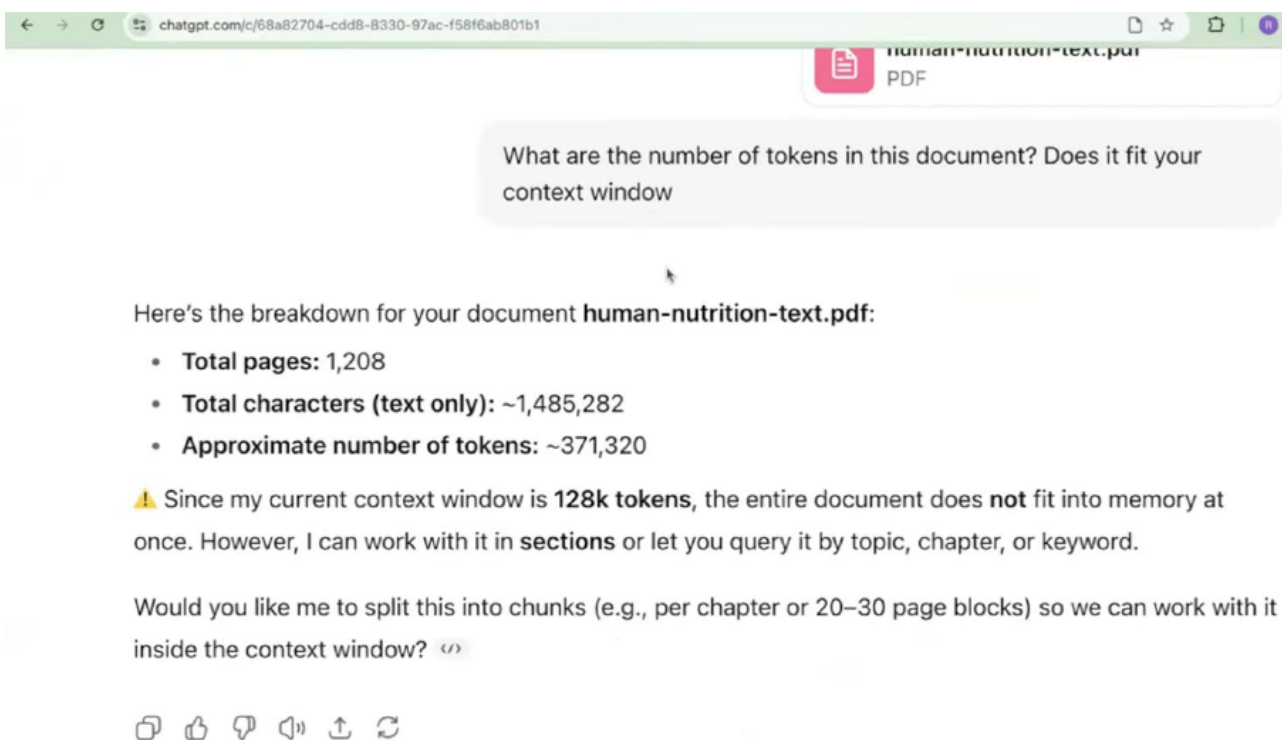
Retriever: finds relevant chunks like an exam index

Augmentor: inserts those chunks into the model's prompt

LLM Generator: reasons using the question + retrieved context



You might now ask, our mind is not capable to read the entire book to answer quickly to the question, but LLMs can right, so what is the use of RAG ? I asked ChatGPT and Google Gemini to answer questions from a 1000 page pdf and here's how it answered.



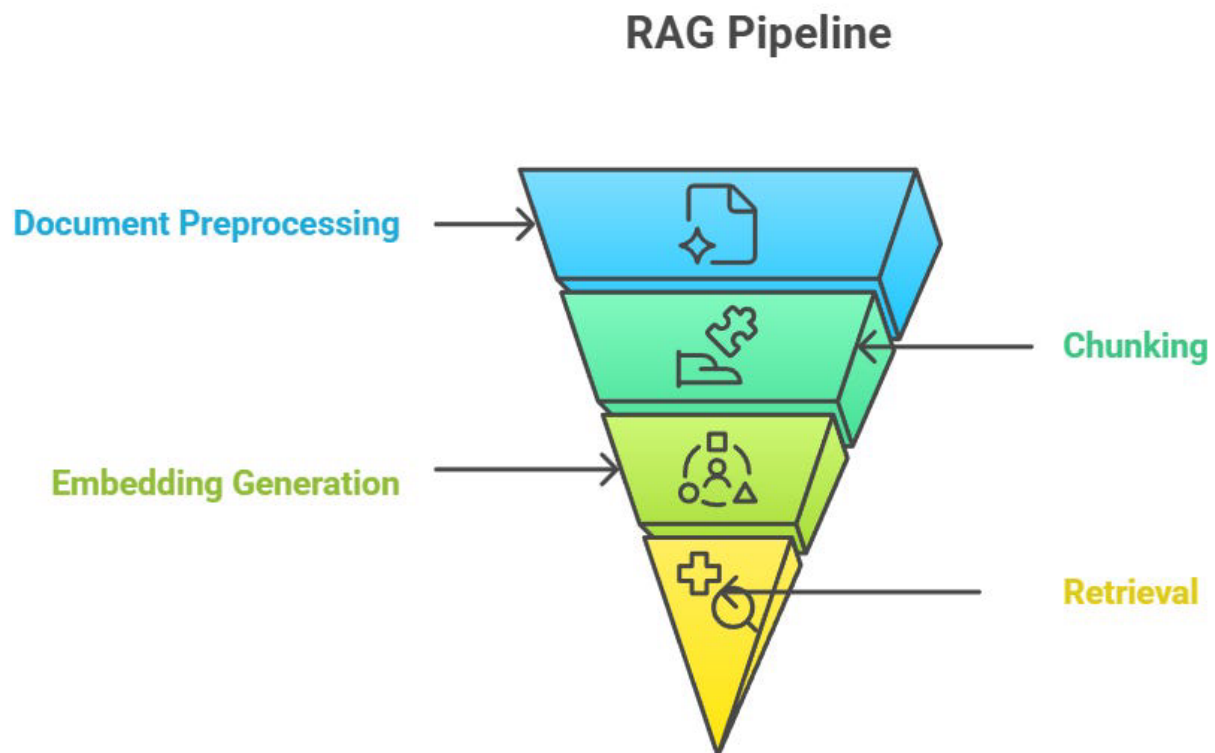
As you can see that Chatgpt was not able to answer as its context window is lesser compared to Gemini. Hence, Google Gemini was able to read and output the question asked. But, here comes the catch: Even though LLMs today have massive context windows and can technically “read” huge amounts of text, there are three major limitations that make RAG still essential.

Even though LLMs today have massive context windows and can technically “read” huge amounts of text, there are two major limitations that make RAG still essential. - Cost explodes with large context windows: LLMs charge per input + output token. Sending a 200K-token PDF into the model every time is insanely expensive.

- Attention quality drops as context increases: LLMs lose accuracy and focus when you overload them. RAG narrows the context -> better reasoning. So RAG is valuable and will be in the future too

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

The Four Main Building Blocks of a RAG System



In this article we will talk about Document Preprocessing that is basically Data Ingestion, and Data Chunking and different strategies in chunking. In the next article we will talk about Embedding generation and retrieval.

Data Ingestion

Everyone thinks Data Ingestion is easy “just upload a PDF and extract text” but there are so many complexities involved in this as well, If given a PDF, where would you store it, how would you parse it? How would you deal with images and tables?

There are some powerful libraries that you should know:

PyMuPDF (fitz) Best for mixed PDFs (text + images). Extracts:

text

images

metadata

layout

Docs: <https://pymupdf.readthedocs.io/en/latest/>

But if a page is JUST a scanned image, PyMuPDF cannot read text -> you need OCR.

OCR using Tesseract For scanned PDFs or images:

Reads text from images

Works for scanned PDFs

Not ideal for complex tables or multi-column layouts

Repo: <https://github.com/tesseract-ocr/tesseract>

Docling Extracts:

rows

columns

table schemas

layout

and converts everything into clean JSON

Docs: <https://docling-project.github.io/docling/>

These libraries help you in opening and reading the pdfs, but in some cases when you are dealing with industry level projects, often you do not have the pdfs, you will have to scrape the data from the web. So here are some libraries you would want to know to scrape data from the web.

Firecrawl Fully crawls:

websites

blogs

documentation

knowledge bases

...and converts them into LLM-ready Markdown.

Docs - <https://www.firecrawl.dev/>

Puppeteer Think of it as an automated browser that you can script the same way you would manually click, search, scroll, and interact with web pages. Also lets you scrape only what you want:

tags

headers

titles

selected divs

links to all PDFs

and even download 500 PDFs automatically

Docs - <https://pptr.dev/>

Data Chunking

Once your documents are cleaned and extracted, the next question is: "How should I split the text so the LLM can retrieve the right information?"

Chunking matters because:

too large = irrelevant info

too small = lost context

badly aligned = wrong retrieval

There are five chunking strategies, each with specific use cases and each of them have their own pros and cons, lets discuss all of that in this discussion :

Fixed-Size Chunking Fixed-size chunking divides text into chunks based purely on length, for example, every 200 words or every 500 tokens. It simply chops the text based on size rules, which makes it extremely fast but contextually weak.

Pros:

Very fast and easy to implement

Works well for large volumes of short, noisy text

Cons:

Breaks semantic flow (chunks may start mid-sentence)

Context can get lost across boundaries

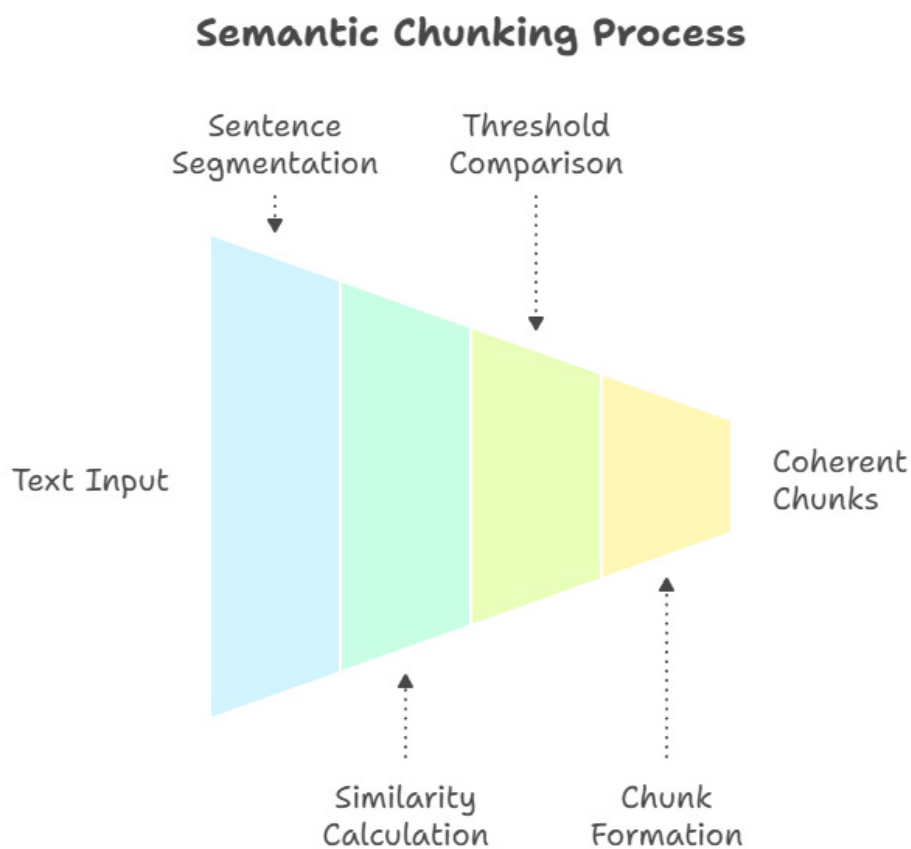
Best For:

Social media data (reddit, twitter etc.)

Logs

Short, unstructured content where coherence is less important

Semantic Chunking



Semantic chunking uses embedding similarity to group sentences that are meaningfully related. Instead of breaking text by size, it breaks based on content. Each sentence is compared to the chunk being built; if it stays semantically similar, it joins the chunk. If it drifts too far than the cosine similarity threshold that we mention then, a new chunk is created.

Pros:

Produces highly coherent, context-rich chunks

Great for deep retrieval accuracy

Cons:

Slow (needs embeddings for every sentence)

Chunk sizes can vary (some can be tiny, others huge)

Best For:

Legal contracts

Policies, compliance docs

Structural Chunking Structural chunking uses the natural format of the document headings, subheadings, sections, tables of contents to define chunk boundaries. It respects the author's intended organization, making chunks more meaningful and human-readable. Example by Introduction, Overview, Results etc.

Pros:

Very clean, interpretable chunks

Fast and consistent

Preserves metadata like titles and sections

Cons:

Sections may vary widely in length hence chunk sizes can vary immensely.

Not ideal for documents with poor or missing structure

Best For:

PDFs with headings

Research papers

Reports, guides, manuals

Technical Documentations

Sliding-window chunking Sliding-window chunking creates overlapping chunks to preserve continuity. Instead of splitting text once, you slide a window across the document for example, a 500-token window moving forward 250 tokens at a time. This ensures that important context that appears near chunk boundaries is not lost.

Example: Chunk 1: tokens 0–500 Chunk 2: tokens 250–750 Chunk 3: tokens 500–1000

Pros:

Maintains continuity across chunk boundaries

Reduces “edge-case” information loss

Improves retrieval for sequential content

Cons:

Higher storage requirements

More redundant data to embed and store

Best For:

Transcripts

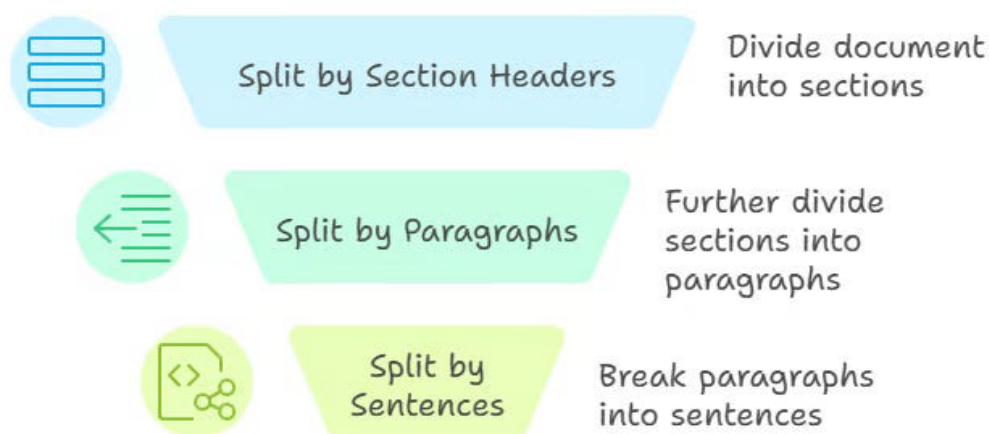
Podcasts

Dialogue/conversations

Any long, sequential content

Recursive chunking

Recursive Chunking Process



Recursive chunking is a hybrid strategy. It first splits the document using high-level structure (e.g., headings). Then, if any section exceeds a maximum size that we mention, it splits that section again using another method (often fixed-size or semantic). This maintains both structure and consistency.

Pros:

Produces consistently sized chunks

Still respects hierarchy and structure

Ideal for large, messy documents

Cons:

More complex to implement

Requires choosing multiple chunking rules

Best For:

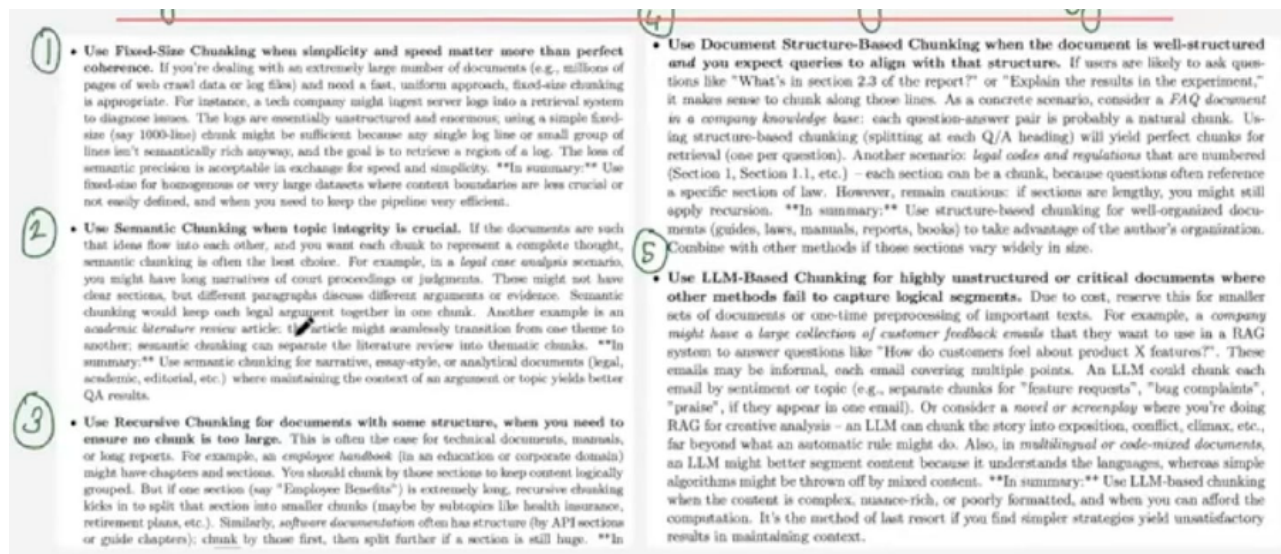
Enterprise-grade RAG

Long, heterogeneous PD

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Which chunking strategy to use and when ?

The image below shows exactly when we should prefer one over the other.



the most important question to ask is: "Does my data need chunking at all?"

Chunking is designed to break down long, unstructured documents. If your data source already has small, complete pieces of information like FAQs, product descriptions, or social media posts, you usually do not need to chunk them. Chunking can even cause problems. The goal is to create meaningful semantic units, and if your data is already in that format, you're ready for the embedding stage.

Once you've confirmed that your documents are long enough to benefit from chunking, you can use the following questions to guide your choice of strategy:

What is the nature of my documents? Are they highly structured (like code or JSON), or are they unstructured narrative text?

What level of detail does my RAG system need? Does it need to retrieve specific, granular facts or summarize broader concepts?

Which embedding model am I using? What are the size of the output vectors (more dimensions increases the ability for more granular information to be stored)??

How complex will my user queries be? Will they be simple questions that need small, targeted chunks, or complex ones that require more context?

How to Optimize The Chunk Size for RAG in Production

Optimizing chunk size in a production setting takes many tests and reviews. Here are some steps you can take:

Begin with a common baseline strategy, such as fixed-size chunking. A good place to start is a chunk size of 512 tokens and a chunk overlap of 50-100 tokens. This gives you a solid baseline that's easy to reproduce and compare other chunking strategies against.

Experiment with different chunking approaches by tweaking parameters like chunk size and overlap to find what works best for your data.

Test how well your retrieval works by running typical queries and checking metrics like hit rate, precision, and recall to see which strategy delivers.

Involve humans to review both the retrieved chunks and LLM-generated responses - their feedback will catch things metrics might miss.

Continuously monitor the performance of your RAG system in production and be prepared to iterate on your chunking strategy as needed.

Already this article has gone sorry for that, in the next article we will learn about Embeddings and Generation part of it. Hope you liked this article, and if you liked it please do leave a like. See you in the next one.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Everything you need to know about Production ready RAG systems: Part 2

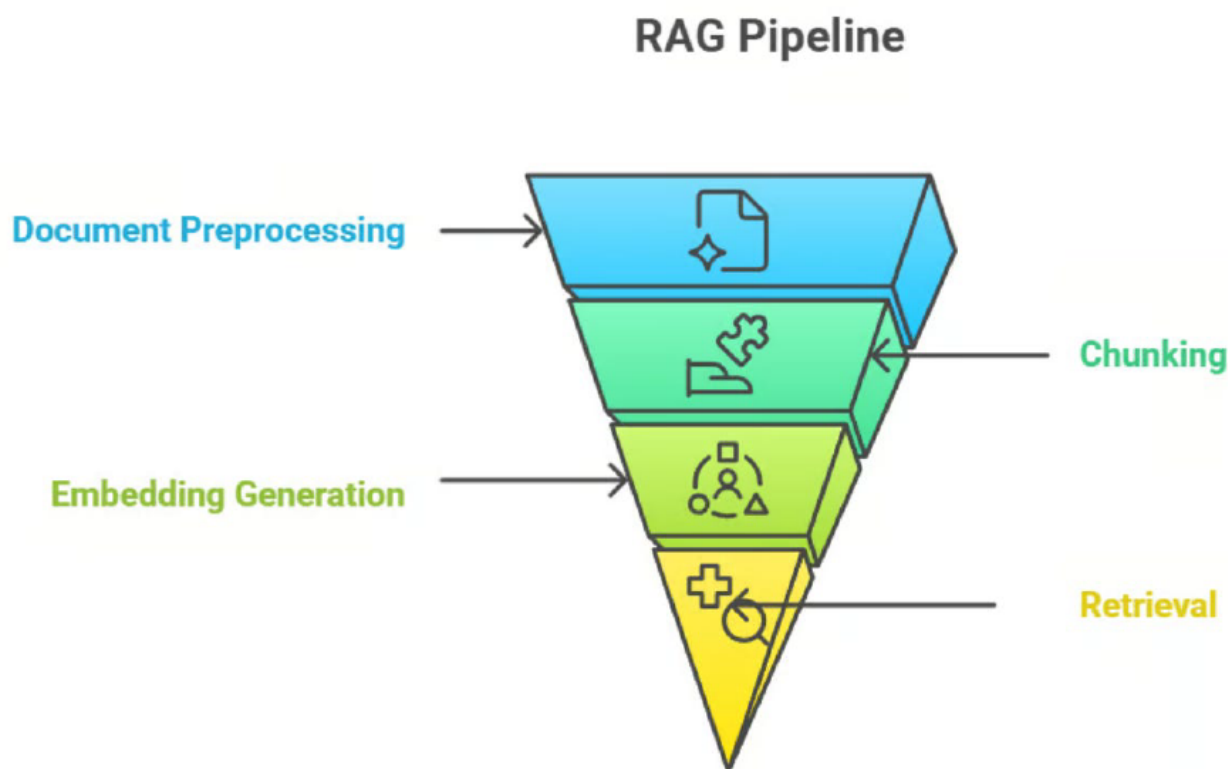
2025-11-23 · Building the semantic layer of RAG: embeddings, vector stores, and high-speed retrieval pipelines

Source: <https://pradyumnachippigiri.substack.com/p/everything-you-need-to-know-about-f99>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/rag-systems-part2>

In Part 1 of this series, we talked about data ingestion and chunking and we learnt how to get raw data into the system and split it into pieces using different chunking strategies.



In this part, we'll cover the last two core pieces of the RAG pipeline:

Embedding generation – turning text into vectors that carry meaning.

Retrieval – using those vectors to fetch the right chunks at query time.

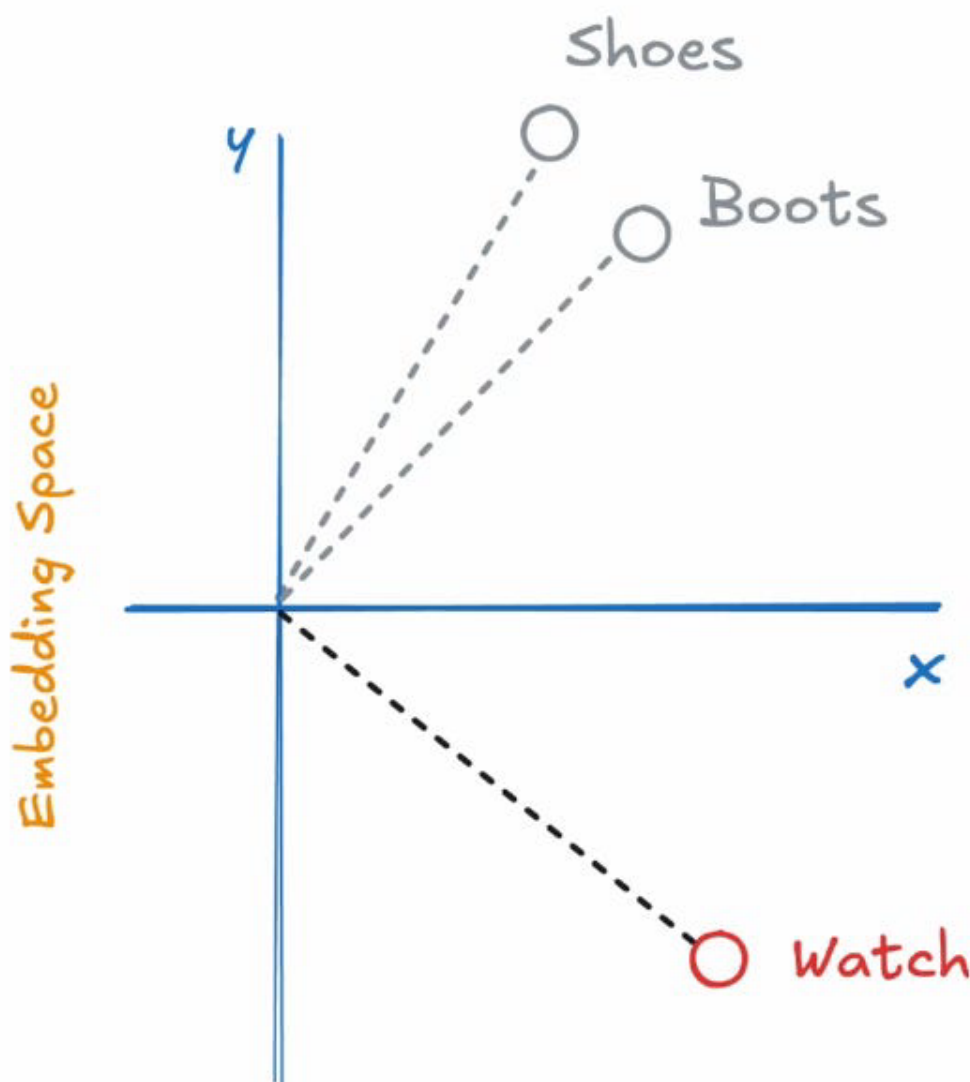
In Part 3 we will talk about the Generation step in RAG, along with different Evaluation strategies to evaluate the setup of the RAG pipeline.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Embeddings: turning text into meaningful vectors

In simple terms, embeddings are numerical representations of data. These numerical representations are not random; they carry some meaning. These embeddings help in capturing the semantic meaning and similarity between data points. For instance, the words “shoes” and “boots” are related in a way that “watch” is not. Embedding is not just for text; it can be applied to images, audio, or even graph data as well. There are multiple embedding techniques, I can make an entire series on that. But for now to understand what makes good embedding, these are the 2 points :

Semantic similarity - This means that words with closer meanings or relationships are closer in the vector space than words that are less related. It uses the concept of cosine similarity.



Dimensions - What should be the size of an embedding vector ? Striking the right balance is key because there are trade offs for example lower lower-dimensional vectors are more efficient to keep in memory or to process, while higher-dimensional can capture intricate relationships, but are prone to overfitting.

From these embedding models families, sentence transformers are one such; sentence transformers enable the transformation of sentences into vectors in vector spaces. They represent sentences as dense vector embeddings. Traditional embedding models used to work on word-level, and sentence transformers work on a sentence level.

Some popular sentence transformers are :

Closed(-ish) models via API

These are the ones you call over an API :

OpenAI (text-embedding-3-large) - OpenAI Embedding Docs

Google Gemini (gemini-embedding-001) - Gemini Embedding Docs

You can't download the model or fine-tune it locally. You just send text -> get back embeddings/vectors. Think of them as black box but convenient.

Open(-weight) models you can run yourself

These are models where you can download the weights and run them locally or on your own cloud:

Sentence-transformers models on Hugging Face(all-MiniLM-L6-v2, all-mpnet-base-v2, etc.)

Gemma, LLaMA, Mistral, etc.

In these kind of models you have more control (privacy, cost, custom infra) You can refer to this link from Huggingface to see the topmost sentence transformers -> Sentence Transformers Leaderboard Embedding models like Gemini, OpenAI can take in huge paragraphs to compute the vectors as they are huge embedding models. (most of these models are multimodals they can deal with images and table etc.) How do you decide which embedding models to use for your RAG usecase ? So to summarise these are the factors to be considered :

Size of the input : If longest input sequences are to be embeded, then chose a model with larger input capacity.

size of embedding vector - Like we discussed larger dimensionalities are more accurate but need more compute and storage.

size of model - Larger models are better but again require more compute and power

open or closed - open models allow to run on your own hardware and closed are easier to setup but require API calls to get the embeddings.

Coming back to RAG, now that we have the chunked documents based on the best chunking strategy for that particular use case, we take the embedding of these chunks. But where do we store these embeddings of the document chunks?

Thanks for reading LevelYouUp! This post is public so feel free to share it.

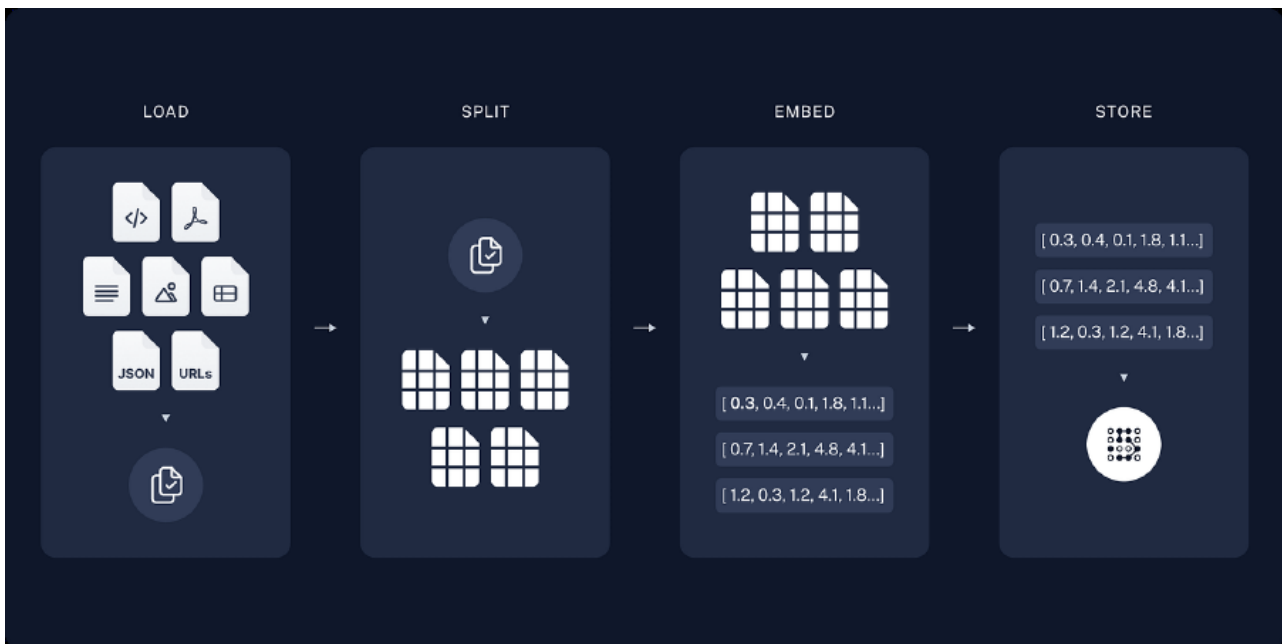
Share

Vector Databases

Well, you may say we can store these in vector databases, thats right, but there are so many vector databases available in the market, which one would you go with ? Each one of them have their own pros and usecases. For a fact even if you pretend vector databases don't exist, a RAG system is still totally doable with just:

an array/tensor of embeddings, shape: (num_chunks, embedding_dim)

some metadata structure that tells you which row corresponds to which chunk of text.



No Vector DB Approach: - persistence to the disk, all embeddings must fit into memory and usually when lesser number of embeddings are involved

Vector DB Approach - db handles indexing, caching etc. - multiple services can read/write concurrently - millions of chunks and embedding vectors are involved.

The Vector DBs that are available in the market are: (chroma as well, missed to include that in the image)

VECTOR DATABASES & LIBRARIES



PINECONE

- Private, Cloud Based, Hugely Popular



MILVUS

- An open-source similarity search engine for embedding vectors



PGVECTOR

- Postgres Vector Search Extension, Useful if you already have data in Postgres



WEAVIATE

- Open Source Vector Database written in Go



FAISS

- A library for efficient similarity search and clustering of dense vectors



QDRANT

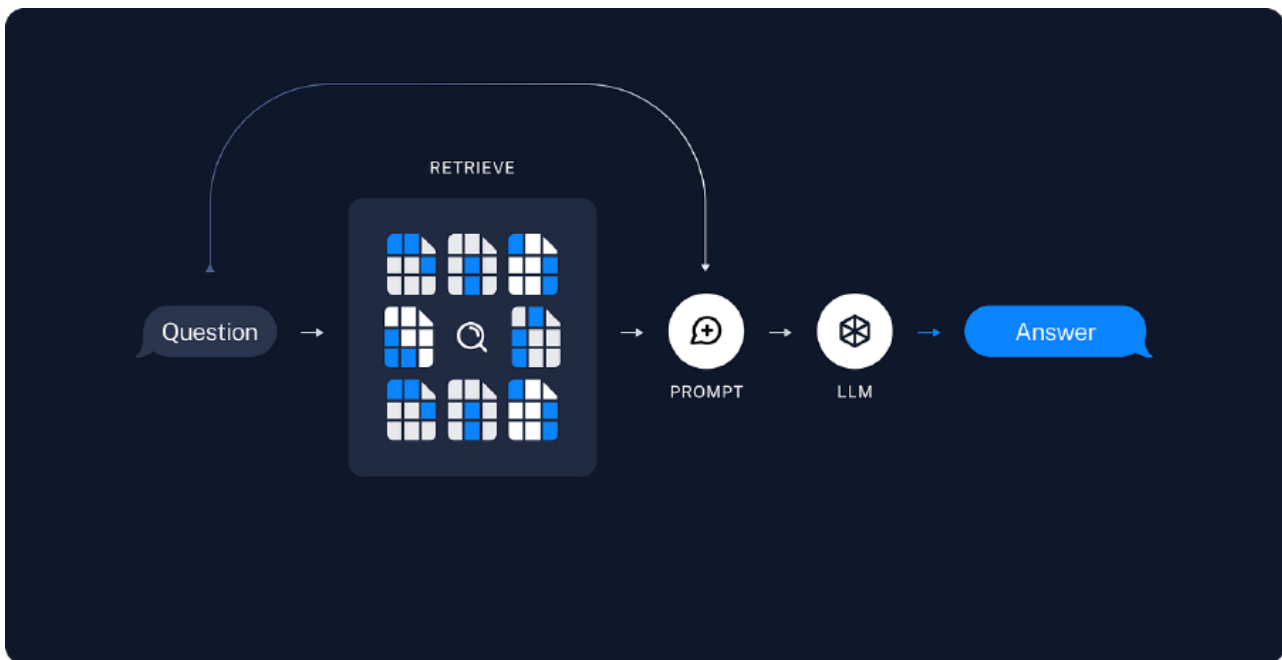
- Open Source Vector Search Engine written in Rust

Why do vector databases need indexing? Every time a user asks a question, we encode that query into an embedding vector on the fly. To answer the question, we need to find the most semantically similar vectors (chunks) from our stored corpus. If we naively compare the query embedding with every stored embedding one by one, it becomes very slow as the number of chunks grows.

Vector indexes solve this by organizing embeddings in a special data structure (HNSW graphs, IVFFlat, etc.) so that we can jump directly to the most promising neighbors instead of scanning everything. That's what makes similarity search fast enough to be usable in production RAG systems. I can talk about different indexing techniques in a different article, the better the indexing strategy, the better and faster the retrieval would be. By default, all of these databases come up with these indexing strategies where we can choose.

For clarity, the query embeddings are not stored anywhere; they are just calculated on the fly, just the document chunks' embeddings are stored in the Vector DB. The same model must be used for both the document and query embedding to ensure uniformity between the two.

Retrieval



Now in the retrieval step :

The user sends a query.

You embed the query into a vector.

The system compares this query embedding with all the document chunks embeddings (via a vector DB indexing).

It retrieves the top-k(should be mentioned in our code) chunks whose embeddings are most similar to the query, using metrics like:

cosine similarity

Euclidean distance

(or other distance metrics supported by the vector store)

These retrieved chunks are considered to be the most relevant context for the user's question and are then passed into the Generation step, which we'll cover in Part 3, along with how to evaluate whether this whole RAG setup is actually doing a good job. I will also try to talk about the different indexing strategies used in these vector databases. Hope you liked this article. If you did, please do shower some love, and do subscribe to this weekly newsletter. Have a nice day! See you in the next one! ?

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Everything you need to know about production ready RAG systems : Part 3

2025-11-28 · Deep dive into the Generation phase in RAG, and widely used RAG evaluation strategies

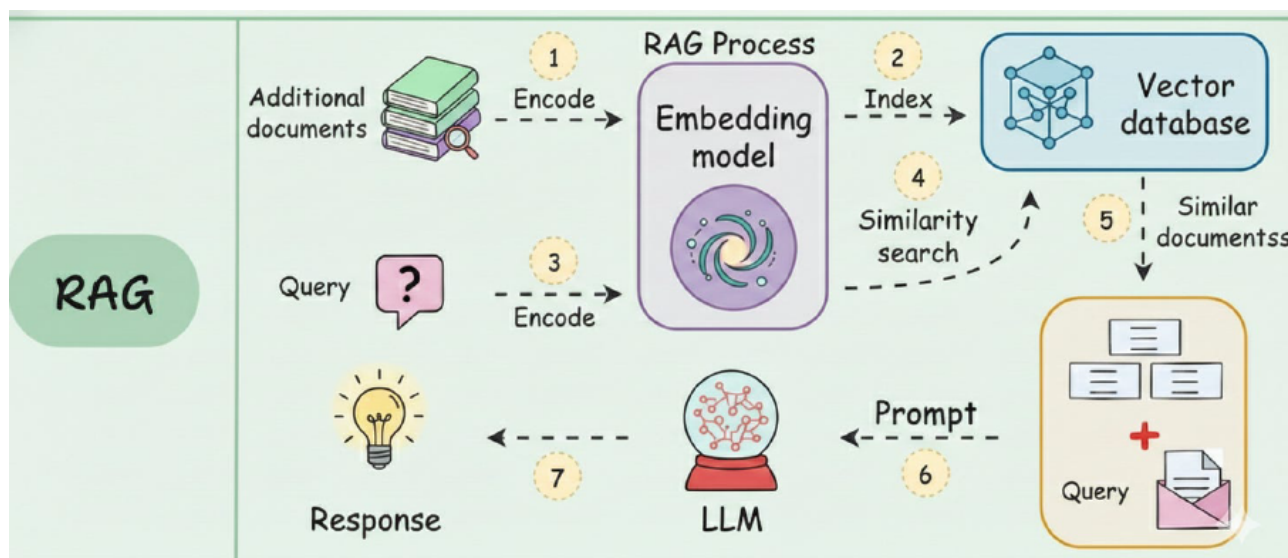
Source: <https://pradyumnachippigiri.substack.com/p/everything-you-need-to-know-about-9be>

In Part 1 and Part 2 we saw the Retrieval and Augmentation phase of RAG. Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/rag-systems-part3>

In this part, we will talk about the Generation part of RAG, along with ways to evaluate a RAG setup.

Generation



By this point in the pipeline, we've already turned everything into vectors: we have an embedding for the user's query and embeddings for all our document chunks. Using similarity search, we've fetched the nearest neighbor document chunks, the pieces of text that are most likely to be relevant to the question. These retrieved chunks become the context window for the next stage: Generation.

In this Generation phase, the retrieved text chunks, along with the initial user query, are fed into a language model with a prompt. The algorithm will use this information to generate a coherent response to the user's questions through a chat interface. This integration allows the LLM to generate responses that are more accurate and contextually relevant, as it can draw from the most current and authoritative information available. This process ensures that the LLM has access to both the context provided by the user's query and the additional, relevant data fetched during the retrieval phase.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Choosing the Model: Local vs API

Here, as an engineer, you have to make a decision: what type of LLM are you going to use? There are two core questions you should ask yourself:

Do you want to run it locally?

If yes, how much compute and memory can you realistically dedicate to it?

If you want the best possible performance with minimal infra hassle, you'll likely start with a hosted API such as GPT or Claude. The trade-offs:

Pros: strong model quality, easy to integrate, no infra to maintain.

Cons: You're sending data to a third-party API (the data is leaving your system), and you pay per token.

If you want to run models locally (or in your own VPC), the second question becomes more important:

What hardware do you have (CPU/GPU, RAM)?

What latency and throughput do you need?

What's your budget for infra?

Your answer to these will determine whether you use a small/medium open-weight model locally, or a large hosted model via API.

Why Prompting Matters in RAG

Writing effective prompts becomes a very important aspect in RAG as well because without effective prompting, even the most relevant retrieved information might not be used correctly by the LLM.

The key reasons prompting matters in RAG are:

Avoiding Hallucinations: This is perhaps the most critical role. A well-crafted prompt explicitly instructs the LLM to only use the information provided in the context, preventing it from fabricating details or drawing upon its general knowledge when the context is insufficient.

Generating Custom Outputs: Prompting allows you to define the desired behavior, personality, tone, and style of the LLM's response. Want a friendly, concise answer or a detailed, formal one? Prompting makes it possible.

Setting Up Constraints and Rules: You can use prompts to enforce specific rules, such as prohibiting the mention of certain topics or requiring a particular format (like bullet points or JSON)

Almost every generation prompt in a production RAG system is some combination of these 4 layers:

Role / System message -> "Who are you?"

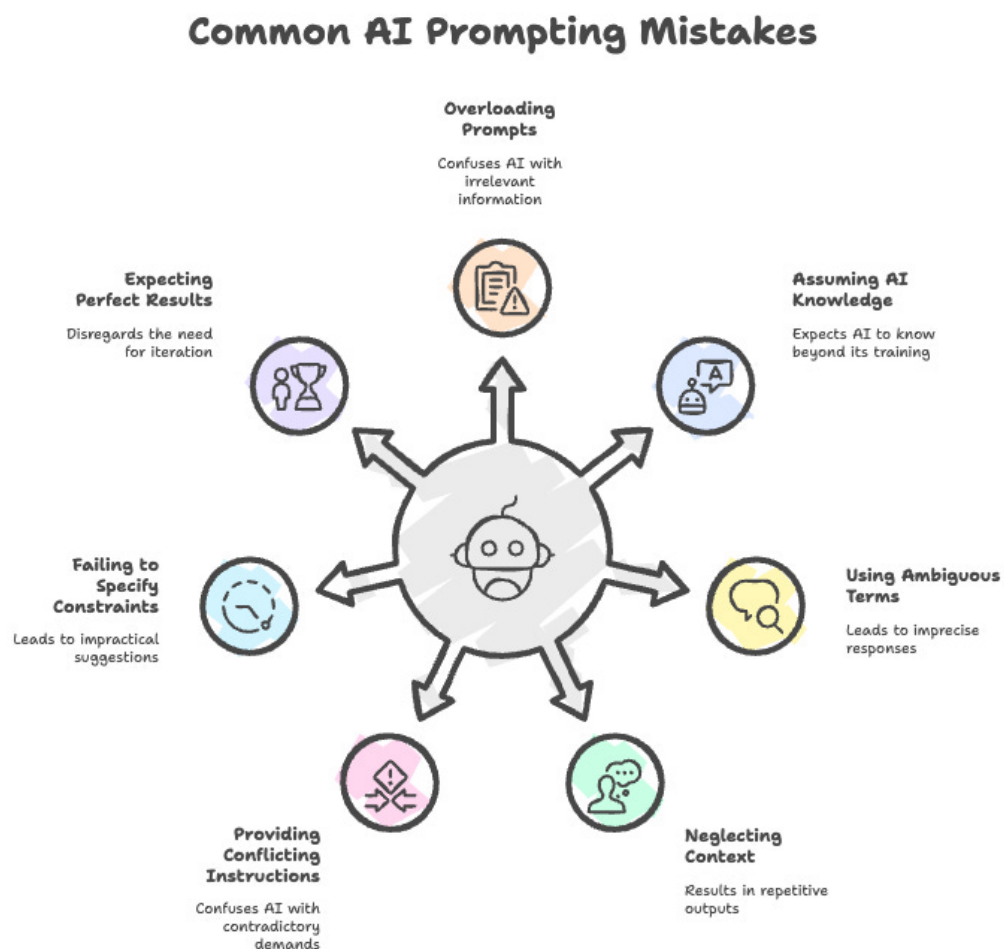
This is your guardrail layer. It's your best defense against hallucinations.

Task instructions -> "What should you do right now?"

Context -> retrieved chunks / "Here is what you're allowed to see" Helps in labelling the chunk for citing it later on if needed.

User query -> the actual question or request

It would be even better if you provide with 1/2 examples so that the LLM can understand better. Once a good prompt is prompted, the LLM outputs the required answers, and our RAG is now fully working to our documents and usecase. Some of the common mistakes in prompting you can see from the image below:



Made with Napkin

But we also need a way to test out this RAG setup of ours,

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

RAG Evaluation pipeline

Evaluating a RAG pipeline helps in evaluating and understanding how our RAG is performing, it involves multiple steps which industries currently follow to evaluate their enterprise RAG.

Step 1: Build a small, high-quality test dataset

Before metrics, you need data:

Collect a set of questions your system should handle

For each question, store:

Ground-truth answer

Relevant source passages/document IDs

Even 30–100 good examples are enough to start. This becomes your RAG test set.

Step 2: Evaluate Retrieval – “Did we fetch the right context?”

Before you judge the final answer, you want to know: “Is my retriever even giving the model the right stuff to work with?”

So, for each question in the test set:

Run it through the retriever (but don’t call the LLM yet).

Collect the top-k chunks that come back.

Look at those chunks and ask:

If I were the model, could I answer this question from these chunks?

There are two ways to do this:

Manual check (great in the early stages): Skim a subset of questions and their retrieved chunks and judge them yourself:

“Yes, this chunk clearly answers the question.”

“No, this is just boilerplate/irrelevant.”

Automatic check (LLM-as-judge): Use another model to score relevance. For each (question, chunk) pair, you can ask: “Given this question and this chunk, how relevant is this chunk to answering the question? Score from 1 (not relevant) to 5 (highly relevant).”

This step isolates retrieval from generation:

If retrieval is weak, the LLM is basically guessing, no matter how good your prompt or model is.

If retrieval looks good but the final answers are still bad, that’s a strong signal that the problem is in the Generation step (prompt design, model choice, or output constraints), not in the retriever.

Step 3: Evaluate Generation – “Is the answer grounded & correct?”

Now we plug in the LLM and evaluate the answers themselves.

For each question:

Run full RAG: query -> retrieval -> generation.

Get the final answer.

Compare answer against:

The context (for groundedness).

The expected answer (if you have one).

Key dimensions:

Faithfulness / Groundedness - Does the answer stick to the provided context, or is it making stuff up?

Relevance - Does it actually answer the question?

Correctness - If you have a expected answers, how close is it?

Completeness - Does it cover all important parts of the query (not just one slice)?

Again, you can:

Do this manually at first,

Then automate with an evaluator model ("LLM-as-judge"), which you prompt like: "Given the question, the context, and the answer, rate:

Faithfulness: 1–5

Relevance: 1–5

Completeness: 1–5 And explain briefly why."

You can store these scores and take averages across the datasets or there are frameworks just for evaluation built for example like RAGAS.

Step 4: Measure Formatting & Contract Compliance

For production apps, it's not enough that the text is correct; it must also:

Return valid JSON when you ask for JSON.

Respect length constraints ("< 200 words").

Use the right tone or structure (e.g., always bullet points).

So you add checks like:

"Is the output valid JSON?" -> yes/no

"Does it contain all required fields?" -> yes/no

"Does it exceed X tokens?" -> yes/no

These can be fully automated with simple scripts.

Step 4: Close the loop and compare different configurations

Now the fun part: you can experiment and compare setups:

Embedding models: model A vs model B

Chunk sizes: 500 tokens vs 1000 tokens

Top-k: k=3 vs k=10

LLMs: GPT-4.x vs open-source model

Prompts: strict vs more relaxed

For each configuration, run:

Retrieval eval -> get relevance/recall metrics.

Generation eval -> get groundedness/correctness/completeness.

Formatting checks -> pass/fail.

In practice, you don't have to build all these evaluation metrics from scratch. There are dedicated frameworks such as RAGAS, TruLens, LangSmith, and Arize Phoenix that use LLM-as-judge-style evaluators to score both retrieval quality (context relevance/recall) and generation quality (faithfulness, answer relevance, hallucinations). RAGAS is one of the most popular RAG-specific libraries.

There are multiple evaluation frameworks out there along with numerous metrics you can read about them here - <https://www.pinecone.io/learn/series/vector-databases-in-production-for-busy-engineers/rag-evaluation/>.

This brings us to the end of the 3-part RAG series.

I hope you now have a clear picture of how RAG works in production systems: from ingestion and chunking, to retrieval and augmentation, to generation and evaluation.

I'd love to hear how you do it in your company, or if there's anything you think I missed.

Thank you so much for reading. Take care, you wonderful people and see you in the next one. ?

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Semantic Caching

2026-09-14 · How Semantic caching helps in saving Cost and reducing latency in LLM Applications

Source: <https://pradyumnachippigiri.substack.com/p/semantic-caching>

Quick Recap : We already know about traditional caching and how it helps in reducing latency and database load.

When the same query is likely to be asked repeatedly again, the cache stores the result as a pair, where the key uniquely identifies the request and the value is the corresponding response. The next time that exact query comes in, the cache serves the stored answer instead of hitting the database again.

Semantic caching applies the same idea to LLM-powered applications, but with a tiny tweak.

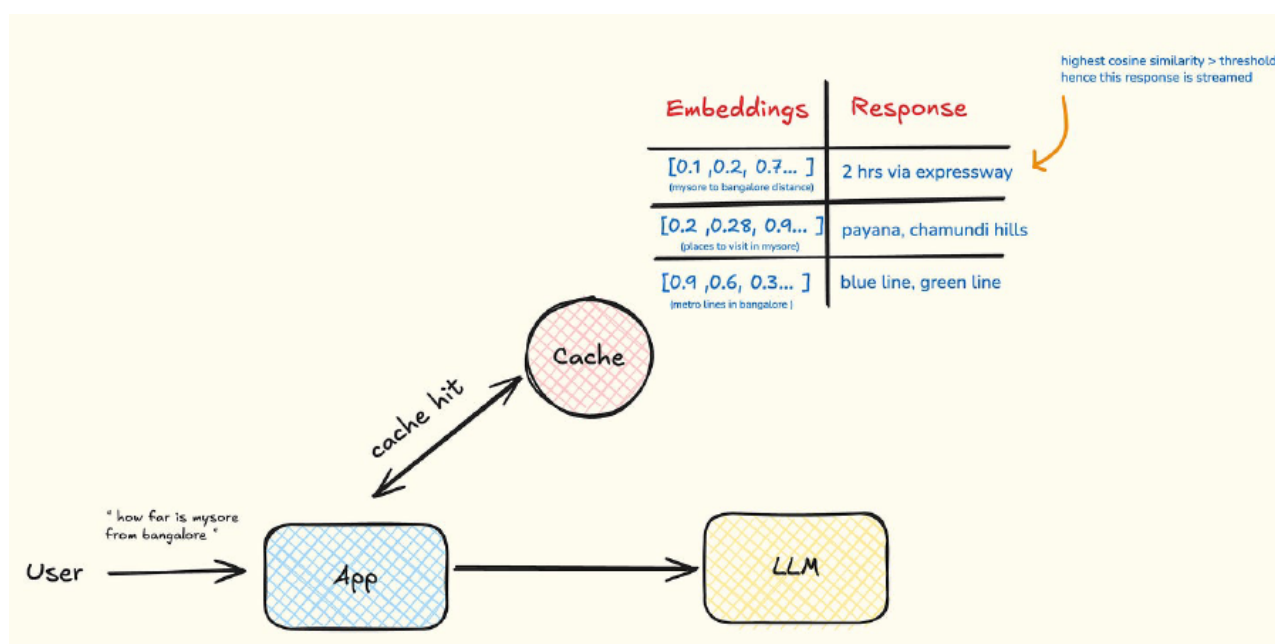
Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

How semantic caching works

Instead of matching queries by exact string, semantic caching matches them by semantic similarity that is by meaning. When a new query comes in, it's first converted into an embedding a vector representation of its meaning. That embedding is then compared against the embeddings already stored in the cache (cache key) using cosine similarity.

If the similarity score passes a set threshold (typically somewhere between 0.85 and 0.95 (we will see why this range in particular late in the note)), the cache treats the new query as equivalent to a previously seen one. So during search if a sufficiently similar cached query is found, the system reuses the previously generated response instead of invoking the LLM again. It not only saved on latency, but it also saved on the token cost of an LLM call.

If there is a cache miss then our app calls the LLM and then stores the response in the cache and then returns the response to the user.



The search that happens here is a vector search that uses approximate nearest-neighbor algorithms like HNSW or libraries like FAISS rather than a brute-force comparison against every cached embedding, which is what keeps the lookup fast as the cache grows.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

The core trade-off: accuracy vs. cost savings

The catch is the threshold. While doing Cosine similarity, we need to decide on what is the best threshold, because here the threshold is what decides if 2 embeddings are close enough or not close enough.

Threshold set too high : the cache becomes becomes strict and the cache barely gets used and we might lose on the cost and latency benefit it was supposed to provide.

Threshold set too low : the cache becomes too easy meaning even if 2 sentences are not similar it will start treating them as similar and would surface wrong answers confidently to the users, which is worst than a cache miss.

A quick note on cache size: Without any proper cache eviction strategies the cache would grow infinitely, and in turn increases the lookup latency as well as the storage cost for the cache.

So what eviction strategies can be used ?

So the same eviction strategies used in traditional caching still apply here, like LRU (drop whatever hasn't been used recently) or TTL (expire entries after a set time), to keep the cache lean and performant.

Where are the embeddings actually generated?

We generate these embeddings on the backend/application side. We utilize an embedding model to convert it into a vector.

Couple of cache services built for this purpose ?

GPTCache (open source library), Redis

Hope you liked this article, if you did then please do subscribe to this newsletter to get weekly notes straight to your inbox.

Article on traditional caching :

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

A Supa Explanation of Supabase: The BaaS Everyone Is Talking About (And Why)

2025-11-12 · In this article we will understand what Supabase is, and why it is making so much positive noise in the market, and how it actually works behind the scenes.

Source: <https://pradyumnachippigiri.substack.com/p/a-supabase-explanation-of-supabase-the>

What is Supabase?

There are many products in this space, if you've used Firebase or Appwrite, you already know the category: BaaS (Backend as a Service).

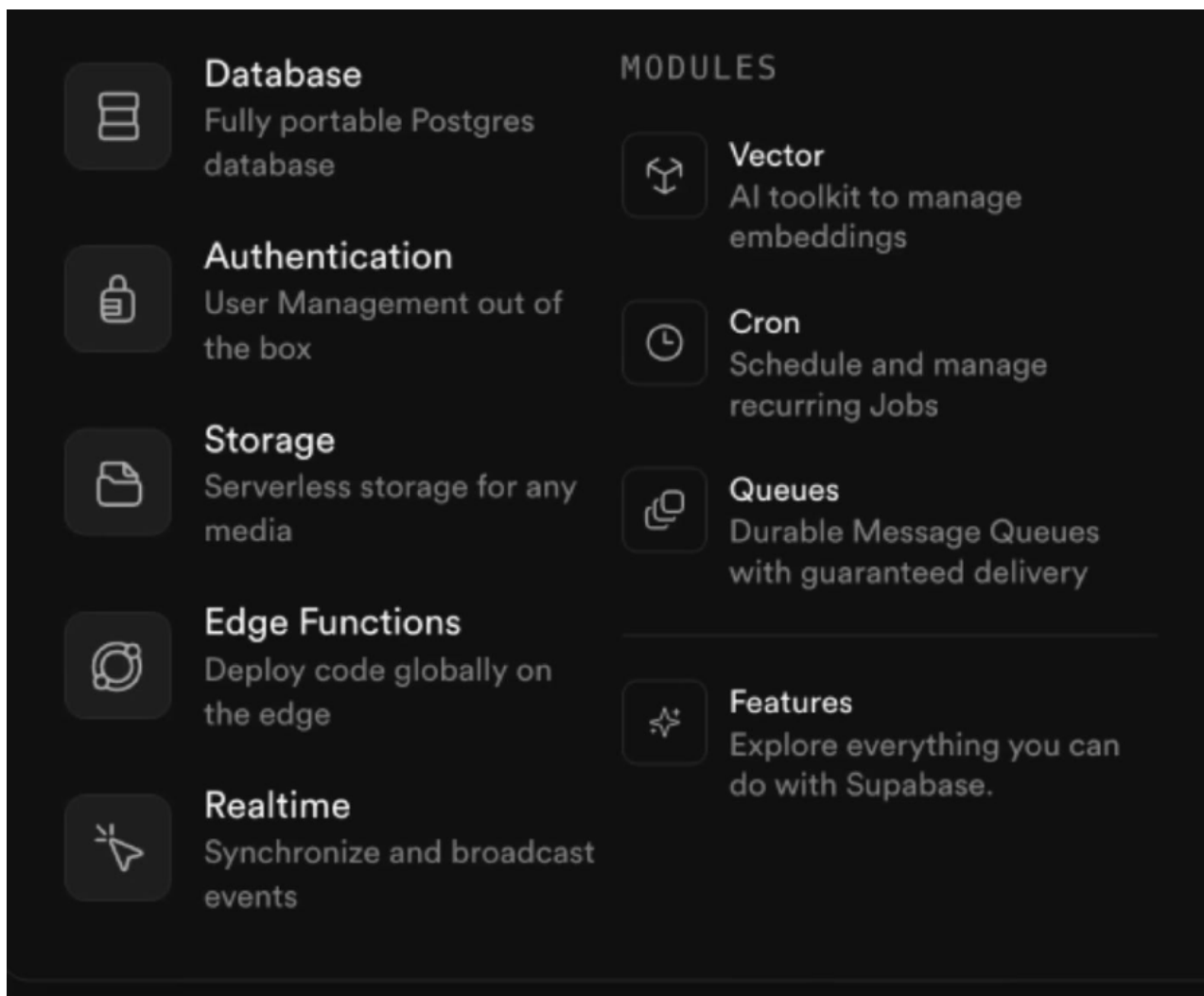
Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

When you build your first few projects, you eventually start seeing a pattern: the same backend pieces keep repeating like auth, database, storage, user management, file handling, etc. Every app needs them, and rebuilding those from scratch each time is exhausting.

That's why tools like Firebase, Appwrite, and Supabase exist. They package these common backend building blocks into ready-to-use APIs, so you can focus on your frontend instead of spinning up servers, databases, and auth systems manually.

This means you can build an entire application in React or Next.js and directly interact with the BaaS no custom backend server needed for most use-cases.

So Supabase is one of these BaaS platforms, but it stands out because it's built on top of PostgreSQL, is open-source, and gives you an SQL-first developer experience. And it handles scale enormously well.



In 2024, Supabase raised \$100M in Series E funding, reaching a valuation of around \$5B. That's massive. But what made this possible? Supabase embraced AI earlier than its competitors and positioned itself as the default backend for AI apps. With support for pgvector, embeddings, and real-time features on top of PostgreSQL, So supabase became extremely attractive for AI startups and developers building LLM-powered applications.

How Supabase works behind the scenes?

Let's say you're building a store app and want to query the name and price of products in the "electronics" category. Here's what your Supabase client code might look like:

```

import { createClient } from "@supabase/supabase-js";

const supabase = createClient(
  "https://[projectId].supabase.co",
  "public_key"
);

async function getProducts() {
  ...
  const { data, error } = await supabase
    .from("products")
    .select("name, price")
    .eq("category", "electronics");
  ...
}

getProducts();

```

This is the supabase query builder from the client-side library where we create the client by providing the project url and a secret key.

If we want to query data ("name" and "price") from the products table where products categorized as electronics.

At first glance, it feels like you're writing SQL right in your frontend but there's a lot happening under the hood.

Supabase converts this SQL like syntax of the client library code into a REST call with query parameters. Something like this :

This request first hits Supabase's API Gateway, which routes it to a service called PostgREST the magic layer that converts your REST query into real SQL for PostgreSQL to execute.

Here's what that REST call becomes internally:

On the backend, Supabase uses something called as the Prepared statements, which basically protects the SQL from SQL Injection meaning Supabase parameterizes it to prevent SQL injection attacks. Our constructed query is wrapped around a template like this :

Parameterized queries not only improve security but also allow for better caching and performance optimization.

Handling Scale with Connection Pooling

If your app suddenly goes viral and thousands of users start querying at once, creating a new database connection for every request would slow things down fast.

That's where connection pooling comes in. Supabase uses pgBouncer, a lightweight PostgreSQL connection pooler, which keeps a pool of open connections ready to be reused. This ensures queries are handled efficiently no matter how much traffic spikes.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Enforcing Security with Row-Level Security (RLS)

Here's where things get interesting.

Now, before PostgREST sends the query to PostgreSQL for it to execute, there is one more important layer that comes in, and that is Row Level Security. Now what's this new thing that everyone's talking about?

The thing is, PostgREST first has to make sure that whoever is executing this query is actually allowed to read the data or not.

And as we discussed, BaaS does not have a backend as we tend to connect the frontend directly to Supabase. If you don't have any backend protection layer, what if data is not secure or gets leaked, especially when you try to talk to the DB directly from the frontend? That's where RLS comes in.

What is Postgres Row Level Security?

Row level security (RLS for short) is a PostgreSQL security feature provided by the open source PostgreSQL database. It allows database administrators to define policies to control how specific rows of data display and operate for one or more user roles.

Row level security restricts database operations on a row level.

For example, the policy below states that only authenticated users can read the data from the table.

×

Create a new Row Level Security policy

Policy Name

Enable read access for all users

Table on clause

public.products

Policy Behavior as clause

Permissive

Policy Command for clause

☒ SELECT

☐ INSERT

☐ UPDATE

☐ DELETE

☐ ALL

Target Roles to clause

authenticated ×

🔒 USE OPTIONS ABOVE TO EDIT

```
1 create policy "Enable read access for all users"
2 on "public"."products"
3 as PERMISSIVE
4 for SELECT
5 to authenticated
6 using (
7   true
8 );
```

When an authenticated user calls `getProducts()`, their JWT token (from Supabase Auth or an external provider like Clerk) is sent along with the request. Supabase verifies that token before executing the query if the user isn't authorized, the query never runs.

Once the SQL query executes, PostgREST receives the result from Postgres, converts it into JSON, and returns it to your frontend. And on the frontend the data would be in the same structured format that you'd expect from any modern API.

name		price
-----+-----		
iPhone 13 Pro		845.99
Sony XM4		199.99
MacBook Air M1		899.99

```
[  
  { "name": "iPhone 15 Pro", "price": 845.99 },  
  { "name": "Sony XM4", "price": 199.99 },  
  { "name": "Macbook Air", "price": 899.99 }  
]
```

Supabase manages the complex backend layers for you from authentication and security to query translation and scaling while letting you stay close to SQL. No boilerplate, no infrastructure stress. You get all the power of PostgreSQL without managing servers, roles, or credentials manually.

No wonder it's rapidly becoming the go-to backend for SaaS and AI startups.

? [Explore the Supabase Documentation](#)

Thank you so much for taking the time to read this article. It takes effort to research, write, and break down complex systems like Supabase into something clear and practical, so if this helped you understand it even a little better, that means a lot.

If you liked it don't forget to share, like, and subscribe to my newsletter.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Stay curious, keep building, and I'll see you in the next article.

Shell Scripting for Devops Beginners: Part 2

2025-11-09 · In this article we revisit some of the most used and most important Linux Commands for interviews, and these Linux commands are also used on a daily basis by every Devops Engineer

Source: <https://pradyumnachippigiri.substack.com/p/shell-scripting-for-devops-beginners-626>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/shell-scripting-part2>

In the last article, we learned about some of the basic Linux commands. In this blog post, we will cover Networking Commands, Process Management commands and loops, and conditional operators in Shell scripting, along with some interview questions.

We forgot to cover a couple of basic commands in the first part, so let's cover that here and then jump to some cool topics.

Also learning about Absolute path and relative path, and that's your homework. Let's dive right in

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Basic Commands

Lets look at some basic operators

> (overwrite): '>' sends the output of a command into a file, replacing the file's existing contents.

Example :

If file.txt exists -> it erases everything and replace with "hello"

If it doesn't exist -> file.txt is created with the content "hello"

>> (append) : '>>' sends the output of a command into a file, but adds it at the end, without deleting existing content.

Example :

If file.txt exists -> "hello" is added at the bottom

If it doesn't exist -> it is created with "hello" in it

We will read about other operators like ('|') pipe.

Networking Commands

Process Management

Commands & Descriptions	Options	Descriptions
<code>ps</code> → Display running processes	<code>-aux</code> <code>-ef</code>	<code>ps -aux</code> → shows all running processes with detailed information., <code>ps -ef</code> also does the same.
<code>top</code> → Monitor system processes in real-time.		displays system processes and its resource usages (cpu usage, memory usage, pids, load avg, who owns the process etc.)
<code>kill</code> → Terminate a process	<code>-9</code> : forcefully kill the process	<code>kill PID</code> → terminates the process with the specified process ID.
<code>grep</code>	used to search for specific patterns or regular expressions in text files or and display matching lines.	<code>ps -ef grep "amazon"</code>
<code>nproc</code>		shows how many cores of our cpu has
<code>free</code> → shows memory usage	<code>-h</code>	<code>free -h</code> → shows memory usage in human readable format
<code>df</code>	<code>-h</code>	<code>df -h</code> → Human Readable Disk space
<code>du</code>	<code>-sh</code>	<code>du -sh /path/to/folder</code> → disk usage of a particular folder

The above commands are very important in day-to-day development, especially when debugging slow systems or tracking down resource with heavy usage.

What is Pipe()parameter ?

Pipe parameter is used to send the output of the first commands to the second commands.

if `ps -ef` returns all the processes then if we do `| grep "google"` it will give us the processes matching the given "pattern" Google.

Now, say if i want to retrieve only a specific column after finding the entire matching sentences. Like if i want the pid of the "google" senence then what would you do ?

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

What is awk ?

awk is a command-line tool and mini programming language for working with text, especially column-based data (like output of `ps`, `df`, logs, CSV-ish stuff).

It reads input line by line

Splits each line into fields (`$1`, `$2`, `$3`, ...)

If pattern matches, it runs the { action }

Say if i want to Print a specific column for example: show only the 1st column of df -h:

\$1 -> first column

\$2 -> second column, etc.

\$0 -> whole line

Basic Conditions and Loops

To understand this, let's see some interview questions, so that it'll be easier for us to understand how these conditional operators and loops work.

Print numbers from [1, 30] divisible by 3

Without `$(( ))` Bash will treat numbers as plain text, not math., and

`[]` is used for conditional checks.

Operator	Meaning
<code>-eq</code>	equal to
<code>-ne</code>	not equal to
<code>-gt</code>	greater than
<code>-lt</code>	less than
<code>-ge</code>	greater than or equal
<code>-le</code>	less than or equal

Print the number of 'S' in MISSISSIPPI

Sort a file alphabetically

And that's a wrap. That brings us to the end of the Shell Scripting for DevOps Beginners series.

These concepts are fundamental for DevOps, SRE, Cloud, and Backend roles. If you found this article helpful, please consider leaving a like and sharing it with your friends. And if you want more such content delivered straight to your inbox, make sure to subscribe.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.